

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
1 ТЕОРЕТИЧЕСКИЙ ОБЗОР	9
1.1 Актуальность и практическая значимость	9
1.2 Способы реализации.....	9
1.2.1 Реализация с помощью MediaPipe	10
1.2.2 Реализация с помощью сверточных нейронных сетей или трансформеров.....	12
1.3 Трансформеры	14
1.3.1 Attention is all you need.....	15
1.3.2 An image is worth 16x16 words: transformers for image recognition at scale.....	18
1.3.3 Swin Transformer: Hierarchical Vision Transformer using Shifted Windows	19
2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	24
2.1 Задача распознавания языка жестов	24
2.2 Структура данных для обучения модели.....	25
2.3 Технические средства для сбора данных	26
2.4 Анализ ограничений набора данных.....	28
3 ПОДБОР ПОДХОДЯЩИХ АРХИТЕКТУР НЕЙРОННЫХ СЕТЕЙ И ИХ ОБУЧЕНИЕ.....	35
3.1 Архитектура на основе ключевых точек, использующая Random Forest	35
3.2 Архитектура LSTM для последовательности ключевых точек	37

3.3	Архитектура на основе видео, использующая визуальные трансформеры	40
4	РАЗРАБОТКА КОНВЕЙЕРА РАСПОЗНАВАНИЯ ЖЕСТОВ ...	44
5	ЭКСПЕРИМЕНТАЛЬНАЯ ПРОВЕРКА	46
5.1	Архитектура на основе ключевых точек, использующая Random Forest	46
5.2	Архитектура LSTM для последовательности ключевых точек	47
5.3	Архитектура на основе видео, использующая визуальные трансформеры	51
	ЗАКЛЮЧЕНИЕ	54
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	55

ВВЕДЕНИЕ

По сей день многие люди, общающиеся с помощью языка жестов, испытывают большие трудности в общении с другими людьми, не говорящими на нем. Для этого было решено разработать нейронную сеть, способную точно распознавать русский язык жестов.

Благодаря быстрому развитию искусственного интеллекта и компьютерного зрения появляется множество возможностей для поиска инновационных решений на стыке между людьми и технологиями. Одним из наиболее перспективных направлений в этом контексте является развитие систем распознавания жестового языка, которые становятся ключевым фактором обеспечения плавного и эффективного общения между человеком и технологическими устройствами.

Визуальные трансформеры являются передовым методом обработки изображений, который использует механизмы внимания. Это открывает новые возможности для точного и эффективного анализа визуальных данных.

Целью данной дипломной работы является разработка нейронной сети, способной с высокой точностью распознавать жесты русской дактильной азбуки.

Был выделен ряд задач, направленных на достижение установленной цели:

- 1) Анализ предметной области и ограничений набора данных дактильной азбуки
- 2) Подбор подходящих архитектур нейронных сетей и их обучение
- 3) Экспериментальная проверка обученной нейронной сети
- 4) Разработка конвейера распознавания жестов

Таким образом данная работа нацелена на разработку эффективной системы распознавания жестов дактильной азбуки с использованием методов глубокого обучения.

1 ТЕОРЕТИЧЕСКИЙ ОБЗОР

В данной главе рассмотрены актуальность и практическая значимость распознавания языка жестов, анализ предметной области, а также способы реализации с обзором аналогичных исследований в этой области. Произведено сравнение архитектур нейронных сетей, а также общая структура данных. Детально рассмотрено внутреннее устройство трансформеров.

1.1 Актуальность и практическая значимость

В наше время, хотя технологии стремительно развиваются, некоторые социальные группы все еще остаются за пределами их влияния. Одной из таких групп является сообщество слабослышащих, которое сталкивается с трудностями в различных ситуациях и может быть неправильно понятым. Например, в некоторых больницах отсутствует сурдопереводчик, что затрудняет взаимодействие между медицинскими работниками и слабослышащими, препятствуя своевременной медицинской помощи. К сожалению, это не единственная проблемная сфера, сфер в которых слабослышащий человек может столкнуться с проблемами очень много.

Последствия глухоты, такие как социальная изоляция и трудности в образовании и трудоустройстве, дополнительно осложняют жизнь этих людей.

В этом контексте системы распознавания языка жестов (SLR) представляют собой возможное решение, способное упростить эти процессы и повысить уровень доступности и комфорта для слабослышащих.

1.2 Способы реализации

Исследования в области распознавания жестовых языков являются важной областью в лингвистике, нейронауках и компьютерном зрении. Хотя конкретных исследований в теме распознавания русского жестового языка достаточно мало, существует множество исследований, посвященных другим жестовым языкам, которые позволяют оценить существующие способы реализации систем распознавания жестовых языков.

В данном разделе рассматриваются передовые способы реализации систем, решающих задачу распознавания изолированных жестов. Все

исследования, представленные в данной главе, являются актуальными, так как выпускались не более двух лет назад.

При разработке систем распознавания языка жестов очень важно использовать современные технологии компьютерного зрения и анализа движений. Одной из таких технологий является библиотека MediaPipe[9], разработанная компанией Google.

1.2.1 Реализация с помощью MediaPipe

MediaPipe – это кроссплатформенный фреймворк, который предоставляет инструменты для разработки приложений, позволяющий анализировать множество ключевых точек тела и лица.

Одним из компонентов MediaPipe является модуль для распознавания жестов рук, который называется MediaPipe Hands, он способен определять 21 ключевую точку кисти в трех измерениях. Ключевые точки представляют собой координаты, которые описывают основные точки руки, образуя всю кисть. Эти точки могут быть использованы для определения положения и формы руки, а также для распознавания различных жестов. Каждая ключевая точка представлена тремя координатами: X, Y и Z. Координаты X и Y указывают положение точки на двумерном пространстве (то есть по горизонтали и по вертикали), в то время как координата Z представляет глубину точки в пространстве (отдаление точки от камеры).

Это позволяет MediaPipe создавать достаточно точные трехмерные модели рук, которые могут быть использованы для обучения модели машинного обучения.

Модель ориентира руки представлена на рисунке 1.1.

Ключевая идея такого подхода к распознаванию жестовых языков заключается в использовании последовательности кадров из видео. Каждое видео разбивается на отдельные кадры, и из каждого кадра извлекаются ключевые точки кисти с помощью библиотеки MediaPipe. После извлечения ключевых точек из каждого кадра мы получаем последовательность данных,

которую можно использовать для обучения рекуррентной нейронной сети (RNN).

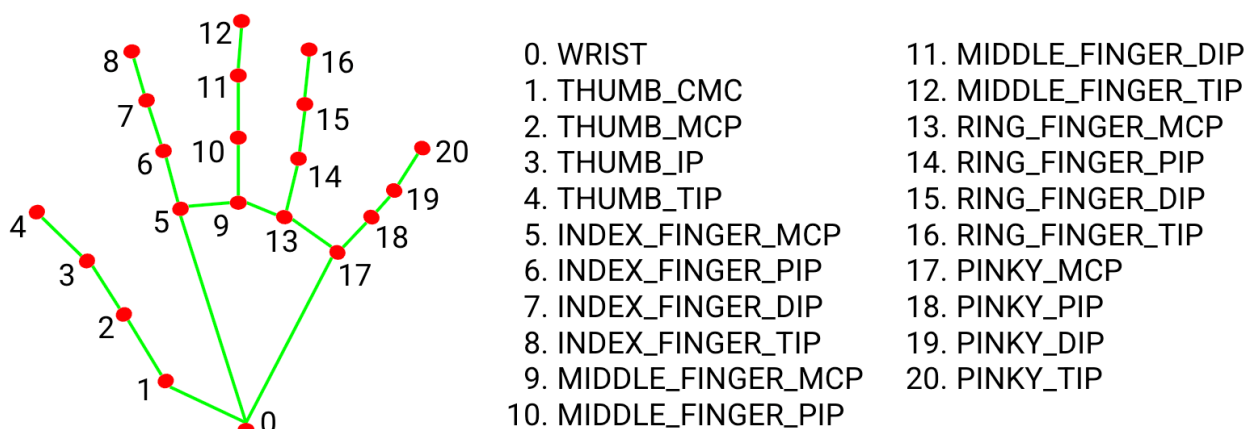


Рисунок 1.1 – Модель ориентира руки MediaPipe

RNN способны улавливать временные зависимости в данных, что делает их идеально подходящими для анализа последовательностей, таких как наши кадры видео. Таким образом, RNN может использовать информацию о совокупности ключевых точек из множества кадров (количество кадров фиксировано) для понимания контекста и распознавания конкретных жестов. Это позволяет системе не только распознавать отдельные жесты, но и понимать их последовательность в контексте всего видео.

Похожий подход используется в исследовании [2].

Исследование посвящено разработке системы распознавания жестов рук в реальном времени, которая использует MediaPipe для извлечения точек захвата руки, LSTM используется для обучения и распознавания жестов.

LSTM – это специальный вид рекуррентной нейронной сети (RNN), отличается она тем, что решает проблему затухающего градиента, которая присуща стандартным рекуррентным сетям.

Система состоит из веб-камеры, которая захватывает движения пользователя и отправляет их в модель для анализа. После обучения модели на наборе данных она способна распознавать и отображать сделанные жесты в виде текста на экране, а также подсвечивать ключевые точки руки (данную функциональность предоставляет фреймворк).

Архитектура системы распознавания жестов представлена на рисунке 1.2.

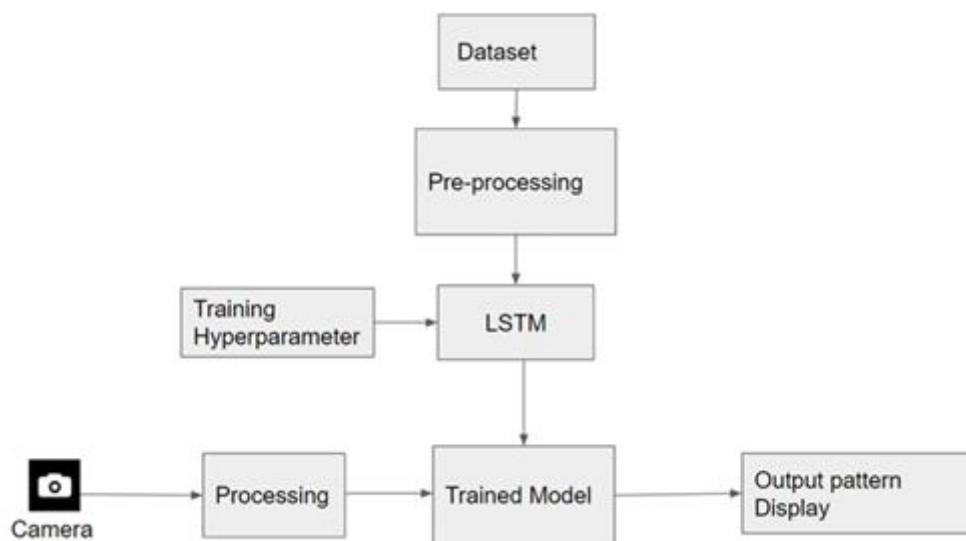


Рисунок 1.2 – Архитектура системы распознавания жестов

1.2.2 Реализация с помощью сверточных нейронных сетей или трансформеров

Сверточные нейронные сети (CNN) и трансформеры являются двумя популярными подходами к решению задачи ISLR.

CNN – это архитектура искусственных нейронных сетей, которая была специально разработана для обработки изображений, а именно для решения задачи детекции (распознавания образов). Они используют математическую операцию свертки вместо обычных мультипликативных весов в своих нейронах. Это позволяет им эффективно обрабатывать изображения, сохраняя пространственные отношения между пикселями.

Трансформеры, с другой стороны, являются моделями машинного обучения, основанными на механизме внимания (attention).

Они были впервые представлены в статье «Attention is All You Need»[3] и с тех пор были успешно применены во многих задачах обработки естественного языка, со временем вытесняя сверточные нейронные сети, трансформеры обходят свёрточные нейронные сети во многих сферах, они

лучше обучаются, однако намного более требовательны к вычислительным ресурсам, так как они очень тяжелые.

При классификации жестов трансформеры могут быть использованы для обработки последовательностей изображений или видео, позволяя модели учитывать временные зависимости между отдельными кадрами.

Механизм решения задачи ISLR представлен на рисунке 1.3.

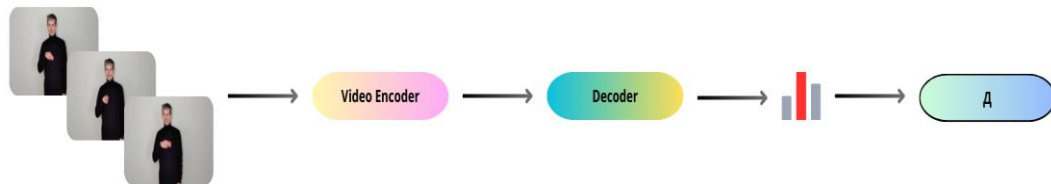


Рисунок 1.3 – Механизм решения задачи ISLR

Вначале, входное видео подается в Video Encoder, который отвечает за извлечение признаков из видео. Эти признаки, также известные как "фичи", представляют собой сжатое представление информации, содержащейся в видео.

Затем, извлеченные признаки передаются в Decoder. Decoder обрабатывает эти признаки и преобразует их в вероятности принадлежности к каждому из возможных классов.

Наконец, функция `argmax` применяется к массиву вероятностей, чтобы определить класс с наибольшей вероятностью. Этот класс и будет результатом классификации.

Архитектура Video Encoder и Decoder представлена на рисунке 1.4.

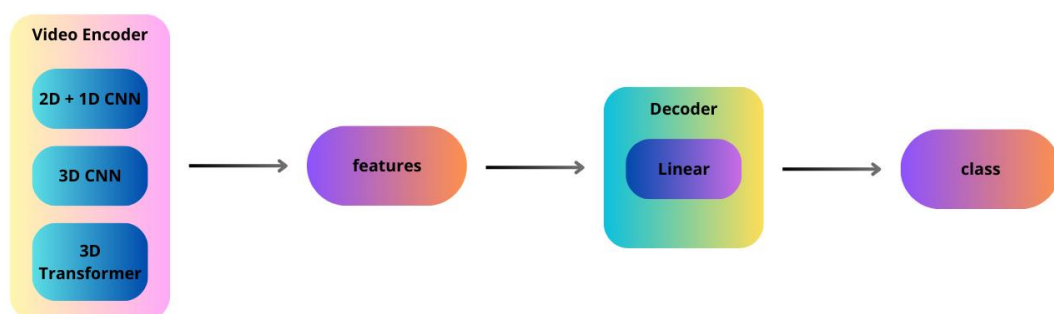


Рисунок 1.4 – Архитектура Video Encoder и Decoder

Подход к обработке видео для классификации жестов предполагает учет временной зависимости в данных. Это достигается несколькими способами:

Можно использовать 2D сверточную нейронную сеть, затем применить 1D свертку к выходным данным. Это позволит модели улавливать контекст последовательности в данных.

Можно использовать 3D сверточную нейронную сеть. Эти модели способны обрабатывать трехмерные данные и могут быть применены непосредственно к видео, так как в них появляется ещё одно пространство, отвечающее за время.

Можно использовать 3D трансформеры. Эти модели, основанные на механизме внимания, могут обеспечить высокое качество классификации. Однако стоит отметить, что обучение таких моделей требует значительных вычислительных ресурсов.

После извлечения признаков из видео, следующим шагом является классификация этих признаков.

Это достигается с помощью линейного слоя, который преобразовывает последовательность признаков в последовательность классов. Каждый класс представляет собой вероятность, которая указывает на уверенность модели в том, что данный набор признаков соответствует определенному классу.

Таким образом, на основе этих вероятностей, мы можем определить, какой класс был предсказан моделью.

1.3 Трансформеры

В данном исследовании особое внимание уделяется использованию трансформеров, поэтому рассмотрим их устройство подробнее.

Перед тем, как говорить о трансформерах, стоит отметить какие проблемы предшествующих моделей решили трансформеры.

На тот момент, для задач, которым нужно было учитывать контекст, активно использовались RNN модели, но вот какие в них недостатки:

- 1) Не поддерживают эффективное параллельное выполнение задач

2) Часто страдают от затухающих и взрывающихся градиентов, что делает обучение моделей с очень длинными входными последовательностями проблематичным

1.3.1 Attention is all you need

Как упоминалось выше трансформеры были впервые представлены в статье «Attention is all you need»[3], но что же нового нам представили трансформеры?

Архитектура трансформера представлена на рисунке 1.5.

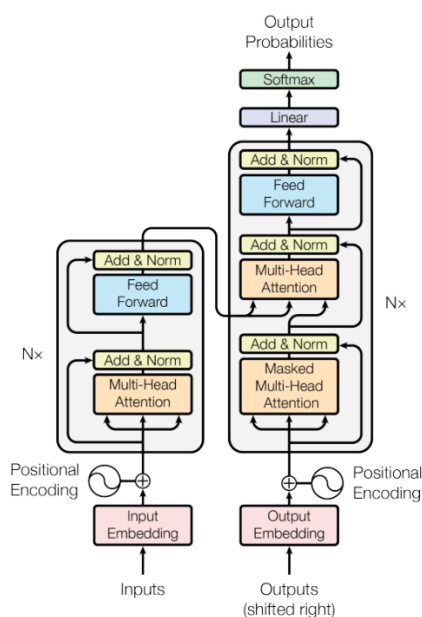


Рисунок 1.5 – Архитектура трансформера

Стоит отметить, что в трансформере нет рекуррентных и свёрточных слоёв, есть исключительно attention. Модель работает быстрее, чем модель Encoder-Decoder. Она смогла достичь хороших результатов в задаче перевода с английского на немецкий. И всё это благодаря концепту self-attention.

Механизм self-attention определяет значимость каждого элемента входной последовательности, при этом не требуется сохранение состояний, это решает проблемы с RNN, описанные выше.

Функция attention — это процесс, который связывает query с набором key-value, генерируя выходные данные. Результат получается вычислением взвешенной суммы значений, где каждый вес определяется на основе совместимости query и соответствующего key.

Масштабированное скалярное произведение внимание представлено на рисунке 1.6.

Scaled Dot-Product Attention

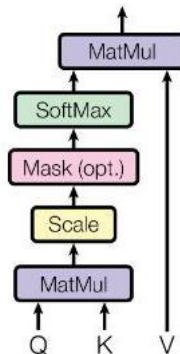


Рисунок 1.6 – Масштабированное скалярное произведение внимания

Входные данные включают в себя queries и keys одного размера (d_k) и значения другого (d_v). Сначала вычисляются скалярные произведения между query и keys, после этого полученные значения делятся на $\sqrt{d_k}$ для масштабирования. Ну а в конце применяется функция softmax, которая преобразует скалярные произведения в веса.

Для детального изучения данная формула представлена ниже.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (1)$$

После знакомства с масштабированным скалярным произведением можно вводить понятие «многоголового» attention, который и использовался в исследовании.

«Многоголовый» attention представлен на рисунке 1.7.

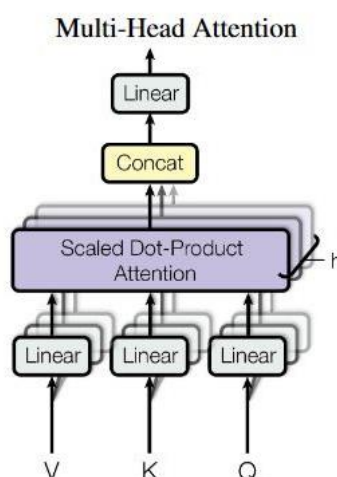


Рисунок 1.7 – «Многоголовый» attention

Как видно из картинки, «многоголовый» attention представляет из себя некоторую распараллеленную версию attention. Наши наборы query, key, value подаются в attention, после чего конкатенируются, преобразуются с помощью линейной функции, что позволяет получить веса.

Следовательно формула имеет следующий вид:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V), \quad (2)$$

$$\text{MultiHeadAttention}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \quad (3)$$

В этой главе была кратко разобрана революционная статья в области глубокого обучения, позволившая вдохнуть новую жизнь в системы искусственного интеллекта.

Далее будут разобраны основополагающие статьи в области визуальных трансформеров, особый упор на которые ставится в данной дипломной работе.

1.3.2 An image is worth 16x16 words: transformers for image recognition at scale

Всё-таки наша задача заключается в компьютерном зрении, а первая созданная архитектура трансформера, как мы можем убедиться, захватывала такую сферу искусственного интеллекта как «обработка естественного языка» (NLP). В какой же момент появились трансформеры, способные составить конкуренцию сверточным нейронным сетям?

Трансформеры в сфере компьютерного зрения использовались максимум совместно со сверточными нейронными сетями, однако это изменилось с выходом статьи «An image is worth 16x16 words: transformers for image recognition at scale»[4].

Что же нового привнесла данная статья?

Разработанная архитектура использовала исключительно трансформеры, минуя использование сверточных нейронных сетей. Более того, получившаяся модель не только требовала меньше вычислительных ресурсов, но и обогнала всех своих конкурентов, использующих сверточные нейронные сети, хотя и пришлось очень сильно потрудиться, обучая её на огромном количестве данных.

Архитектура ViT представлена на рисунке 1.8.

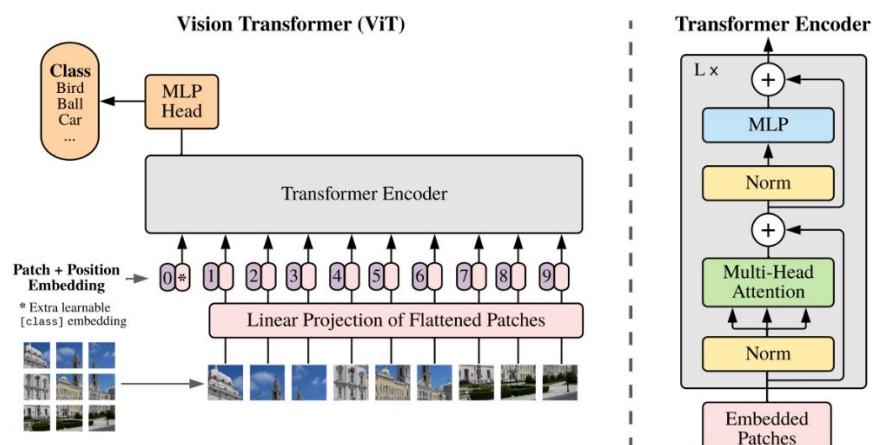


Рисунок 1.8 – Архитектура ViT

Итак, начальное изображение разбивается на квадратные патчи 16x16, которые впоследствии подаются на вход нейронной сети с использованием

Position Embedding. На выходе из трансформера мы получаем вектор, соответствующий CLS-токену, после прохождения через полносвязные слои мы предсказываем класс.

Стоит отметить, что архитектура ViT подразумевает encoder с CLS-токеном, что говорит о том, что это идейный наследник BERT. Данная архитектура была представлена в статье [5], однако на ней мы не будем останавливаться.

Не всё так хорошо и в трансформерах – одной из главных их проблем является то, что они требуют квадратичное число операций и памяти относительно длины входной последовательности. Это связано с тем, что каждый элемент последовательности сравнивается с каждым другим для вычисления весовых коэффициентов. Что для картинок с высоким разрешением чревато огромными затратами по времени, а мелкие детали и вовсе могут быть потеряны уже на первом слое.

1.3.3 Swin Transformer: Hierarchical Vision Transformer using Shifted Windows

Время шло и в статье «Swin Transformer: Hierarchical Vision Transformer using Shifted Windows»[6] представили свою архитектуру, способную бороться с проблемой квадратичного внимания.

Какое же решение они предложили? Давайте ознакомимся с архитектурой, представленной на рисунке 1.9.

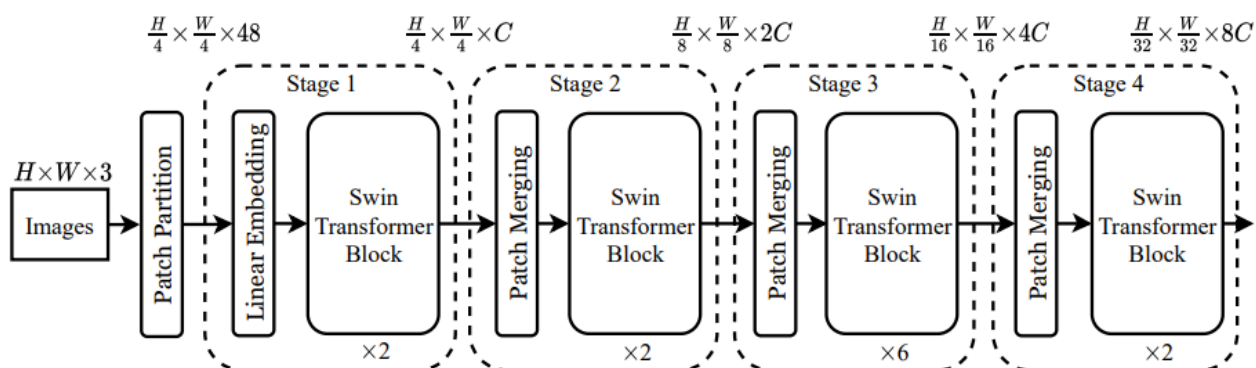


Рисунок 1.9 – Архитектура Swin Transformer

Отсюда можно выделить следующие особенности данной архитектуры:

Итак, первое, что происходит здесь – это мы подаём на вход изображение, которое далее разделяется на 4 части по 4 квадрата. В ViT же использовалось 16x16 вместо 4x4 как раз из-за проблемы квадратичного внимания. Здесь же это будет решаться, и квадратичная сложность уменьшится до линейной. Слой Linear Embedding же не несёт в себе чего-то кардинально нового за исключением использования патча 4x4 вместо 16x16.

Вот что является новым, так это «Swin Transformer Block». Что же вообще такое Swin? Название Swin как ни странно идёт от shifted window (сдвинутые окна). Всё-таки нужно понять, откуда такое название и почему этот блок сдвоенный.

А разобраться в этом поможет иллюстрация, показывающая внутренности Swin Transformer Block.

Она же представлена на рисунке 1.10.

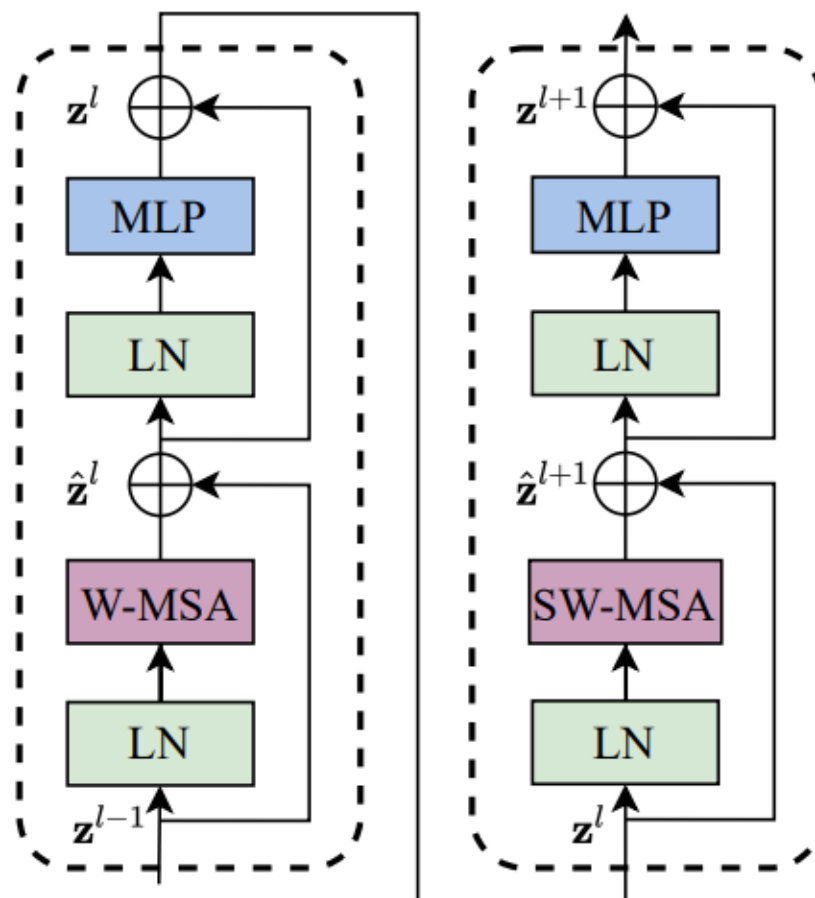


Рисунок 1.10 – Swin Transformer Block

Заметим, что наш Swin Transformer Block похож на Transformer Encoder. Рисунок 1.8 иллюстрирует его.

Видно, что все блоки похожи за исключением блоков W-MSA (Windowed Multi-Head Self Attention) и SW-MSA (Shifted Windows Multi-Head Self Attention). Их мы и рассмотрим подробнее. Это даст нам ключ к пониманию из-за чего удалось снизить сложность с квадратичной до линейной.

Чтобы удостовериться в том, что сложность теперь линейная, представлены следующие формулы:

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C, \quad (4)$$

$$\Omega(W - \text{MSA}) = 4hwC^2 + 2M^2hwC, \quad (5)$$

Здесь hw – это размер входного изображения, а M – это фиксированный размер окна в патчах.

И действительно, теперь сложность алгоритма действительно линейная.

W-MSA – тут нам определяется то самое окно, которое охватывает матрицу патчей 4×4 . Данные окна не перекрываются. Мы указываем патчи внутри каждого окна как входные данные в обычный MSA, после чего получаем результат. Таким образом мы сокращаем сложность с квадратичной до линейной. Это имеет смысл, так как многие из пикселей в разных патчах никак не взаимодействуют друг с другом. Однако может ведь быть и так, что наш объект находится в нескольких окнах одновременно и слой attention не сможет понять, что это за объект. Для того, чтобы решить эту проблему авторы предложили ещё один уровень MSA, который называется SW-MSA.

SW-MSA как следует из названия перемещает это самое окно. Размер сдвига равен половине окна, то есть если у нас окно размером 4×4 , значит мы должны его сдвинуть на 2×2 . При этом в таком случае получится окно 3×3 , которое, во-первых, увеличит в 2.25 раза время обучения, а во-вторых, не даст

нам необходимого эффекта в некоторых случаях (будут зоны, которые всё равно не будут покрыты ни W-MSA, ни SW-MSA).

Наглядно то, что описано выше представлено на рисунке 1.11:

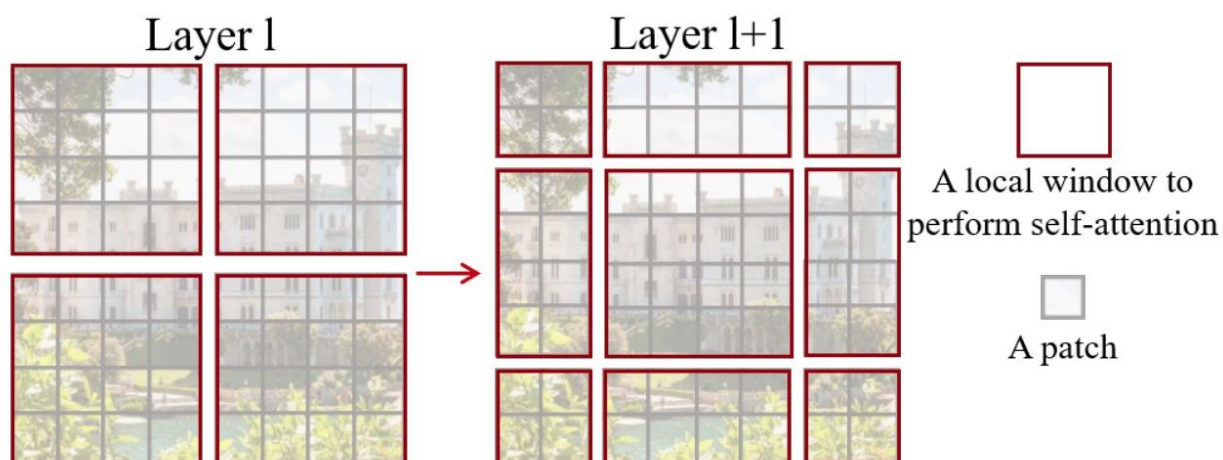


Рисунок 1.11 – Наивный SW-MSA

Для решения проблемы, описанной выше, авторы предложили метод, который предотвращает взаимодействие не соседних окон в изображении при вычислении attention. Этот метод включает в себя циклический сдвиг самого изображения перед вычислением attention. Таким образом не нужно будет вычислять большее количество attention для дополнительных окон.

Данный механизм представлен рисунке 1.12.

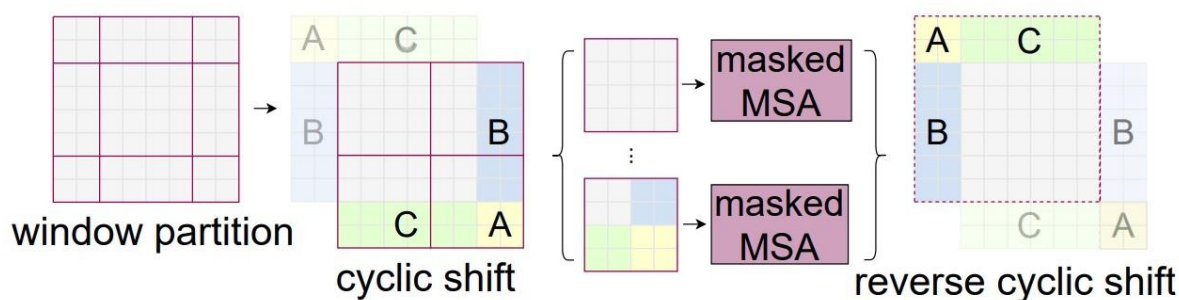


Рисунок 1.12 – Улучшенный SW-MSA

В статье также были представлены некоторые нововведения относительно классических визуальных трансформеров, однако основные моменты уже были описаны в данном обзоре.

Авторы представили несколько моделей, отличающихся количеством параметров (чем больше параметров – тем большая точность и тем большая вычислительная сложность) и размером изображения, поступающего на вход.

Среди всех представленных моделей в силу высоких технических требований для задачи ISLR следует выбрать самую простую модель, таким образом каждый слабослышащий смог получить доступ к данному решению. В иных же случаях требуются намного более мощные вычислительные устройства, чем офисные ноутбуки и смартфоны.

2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

2.1 Задача распознавания языка жестов

Система распознавания языка жестов (SLR) представляет собой технологическую систему, разработанную для автоматического распознавания и интерпретации жестов, используемых в языках жестов. Язык жестов является способом коммуникации, используемым глухими и слабослышащими людьми. Технология SLR преобразует жестовые выражения в текст или аудио речь, улучшая взаимодействие между людьми, использующими язык жестов и техническими устройствами или людьми, не знающими язык жестов. Эта технология находит применение в различных областях, включая образование, медицину, ассистивные технологии и сферу общественных услуг, и многими другими сферами, где очень важно поддерживать контакт с потребителями услуг.

Распознавание языка жестов можно разделить на 3 задачи:

- 1) ISLR – это задача распознавания жестового языка, которая включает в себя анализ отдельных жестов и их классификацию
- 2) CSLR – это задача распознавания жестового языка, которая включает в себя анализ видеопотока и определение последовательности жестов, представляющих слова или фразы на жестовом языке
- 3) SLT – это задача перевода жестового языка, которая включает в себя преобразование жестового языка в письменный или устный естественный язык. Это сложная задача, которая требует понимания контекста и семантики жестового языка, а также умения переводить это понимание в естественный язык

В данном исследовании мы фокусируемся на решении задачи ISLR, которая представляет собой важную и активно развивающуюся сферу в области компьютерного зрения и машинного обучения. Задача ISLR заключается в распознавании отдельных, изолированных жестов на видео, что является ключевым шагом для понимания и интерпретации жестового языка. Мы применяем современные методы машинного обучения и компьютерного

зрения для обнаружения и отслеживания рук, извлечения признаков и классификации жестов.

Главная задача – улучшить точность и эффективность распознавания жестов, что в последствии способствует уменьшению потребности глухонемых и слабослышащих в сурдопереводчиках и широкому применению подобных технологий в сфере услуг.

Жесты в русском жестовом языке намного сложнее, чем нам того бы хотелось и это касается в том числе и дактильной азбуки. Дактильная азбука – это эквивалент хорошо нам знакомой азбуки, за тем лишь исключением, что буквы не проговариваются, а показываются жестами.

Для чего же в языке жестов азбука? Она нужна для определения слов, которые не покрывает язык жестов, например имена. Было бы сложно для каждого имени создавать жест. На самом же деле, достаточно дактилировать имя лишь 1 раз, а далее к человеку обращаются по какому-то признаку фамилии, имени или внешности.

Также стоит отметить, что в дактильной азбуке существует несколько жестов, похожих друг на друга. Это может осложнить обучение модели. Так, например, есть жесты, которые показываются абсолютно одинаково, но отличаются исключительно движением руки.

2.2 Структура данных для обучения модели

Стандартный набор данных для задачи ISLR обычно включает в себя следующие компоненты:

- 1) Видео или изображения жестов: это основной компонент набора данных. Каждое видео или изображение представляет собой отдельный жест. Видео могут быть разной продолжительности, но каждое из них представляет собой один жест. Изображения могут быть статическими или последовательностями кадров, представляющих жест.
- 2) Метки классов: для каждого видео или изображения в наборе данных должна быть метка класса, которая указывает, какой жест

представлен. Метки классов обычно представлены в виде целых чисел или строк.

Пример типичных обучающих данных для задачи ISLR представлен на рисунке 2.1.

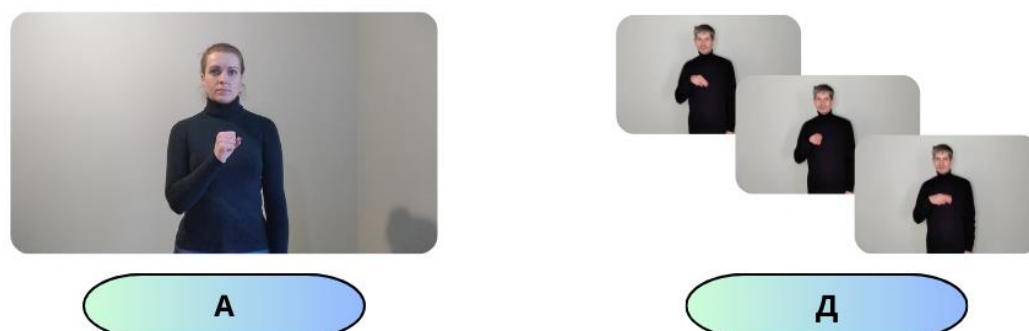


Рисунок 2.1 – Пример обучающих данных и меток

2.3 Технические средства для сбора данных

Технические средства для сбора данных могут быть следующими:

1) Видеокамера

Данный способ имеет наибольшую прикладную ценность, так как далеко не все организации будут закупать какое-то оборудование для сурдоперевода. Для конечных пользователей данная система наиболее удобна, так как для того, чтобы начать пользоваться сервисом достаточно просто включить камеру на смартфоне или любом другом устройстве. Большинство современных исследований используют данный подход к получению информации. В данном исследовании будет использоваться именно это техническое средство.

На рисунке 2.2 представлена камера смартфона.



Рисунок 2.2 – Камера смартфона

2) Специальные перчатки

При данном подходе слабослышащий надевает специальные перчатки, которые считывают положение кисти руки в пространстве, что обеспечивает высокую точность значений ключевых точек руки в пространстве. В таком случае достаточно сложно получить хороший, объемный датасет, захватывающий большое число жестов, поэтому данный вариант практически нереализуем.

На рисунке 2.3 представлена перчатка для определения жестов.

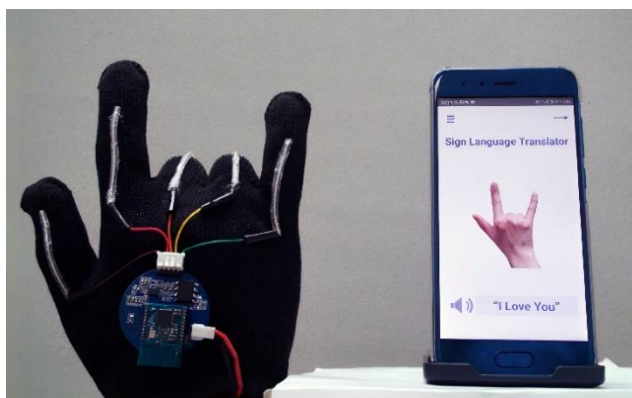


Рисунок 2.3 – Перчатка для определения жестов

3) Камеры с датчиками глубины

В данных исследованиях камеры с датчиками глубины (например, Microsoft Kinect) используются для того, чтобы проще выделить область жеста. Работает это следующим образом: с помощью датчика глубины строится карта глубины, так как кисти рук при общении языком жестов, находятся ближе всего к камере, то мы можем с легкостью обработать изображение, применив к нему фильтр, который позволит с легкостью

получить область кисти руки (так решается задача детекции). Данный процесс реализован в исследовании [1].

На рисунке 2.4 представлен Microsoft Kinect.



Рисунок 2.4 – Microsoft Kinect

2.4 Анализ ограничений набора данных

Начнём с того, что нам представляют создатели набора данных.

Файловая иерархия набора данных представлена на рисунке 2.5.

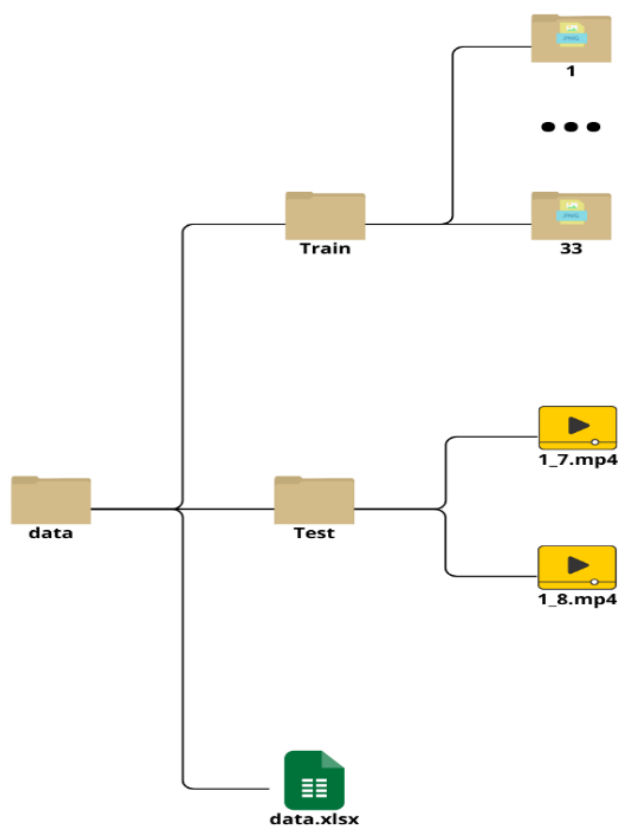


Рисунок 2.5 – Файловая иерархия набора данных

Отсюда видно, что наш набор данных делится на папки «Train» и «Test», а также есть файл аннотаций «data.xlsx».

Папка «Train» состоит из 33 папок, каждая из которых названа числом от 1 до 33. На самом деле это порядковый номер буквы в русском алфавите. В этих папках могут быть как изображения в формате .png, так и видео в формате .mp4. Это связано с тем, что наши жесты могут быть как статичными, так и динамическими, для распознавания которых нужен контекст, поэтому создатели набора данных решили разделить эти данные и видео со статичными данными нарезали по кадрам, а с динамическими оставили как есть. Насколько это правильный подход рассмотрим немного позже.

Папка «Test» состоит из 2-ух видео, на обоих видео показываются жесты по порядку, то есть «от А до Я». Отличаются эти 2 видео тем, что на них разные люди. Для настоящего тестирования получившейся модели данные «тестовые» данные не подходят, так как в них нет никакой разметки, количество людей слишком маленькое, а также жесты показываются совсем не так, как они показывались в датасете. Например, бывают моменты, когда человек специально начинает показывать действия медленнее, что точно не очень хорошо для модели, обученной на определенной скорости показания жестов. В общем результаты тестирования, полученные на этих тестовых данных, не будут нести никакой практической значимости, однако в качестве демонстрационного примера они могут пригодиться.

Что же представляет из себя файл аннотации? Столбцы из файла аннотации представлены на таблице 1.

Таблица 1 – Столбцы файла аннотации

n	file	label	path	landmarks
----------	-------------	--------------	-------------	------------------

Отсюда n – номер строки, file – название файла, label – истинный класс файла, path – путь к файлу, landmarks – ключевые точки.

Пожалуй, самым интересным столбцом здесь является «landmarks». Рассмотрим его подробнее.

Откуда же взялся этот столбец? В главе был разбор функционала библиотеки MediaPipe. В частности, детально описан механизм получения ключевых точек руки. Здесь как раз записаны ключевые точки руки в трёх проекциях (X, Y, Z).

Было решено проверить корректность представленных данных. Процесс проверки выглядел следующим образом. Берем входное изображение с помощью библиотеки OpenCV [13]. Далее берём landmarks (в аннотационном файле точки нормированы) и отображаем эти точки на изображении.

После выполнения данных действий стало ясно, что ключевые точки, собранные создателями набора данных некорректны.

Хотя и ключевая задача, отмеченная в том числе и в названии ВКР подразумевает использование визуальных трансформеров, однако архитектура нейронной сети, использующая ключевые точки руки, также будет рассмотрена в дальнейшем.

Давайте рассмотрим детальнее наши данные. на рисунке 2.6 представлены объем и количество файлов в наборе данных.

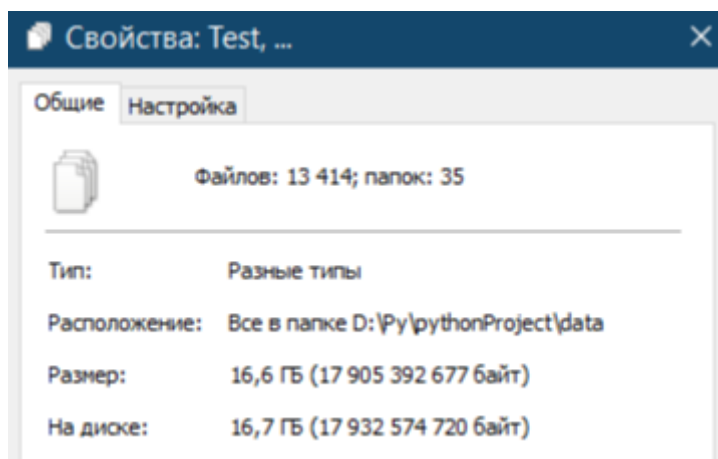


Рисунок 2.6 – Свойства начального набора данных

Как видно из приведенной статистики, сам набор данных занимает 16,6 Гб, представлен в виде 13414 файлов разных типов (видео и изображения). Набор данных не является сбалансированным, точная статистика по тренировочным изображениям и видео представлены на рисунке 2.7 и рисунке 2.8.

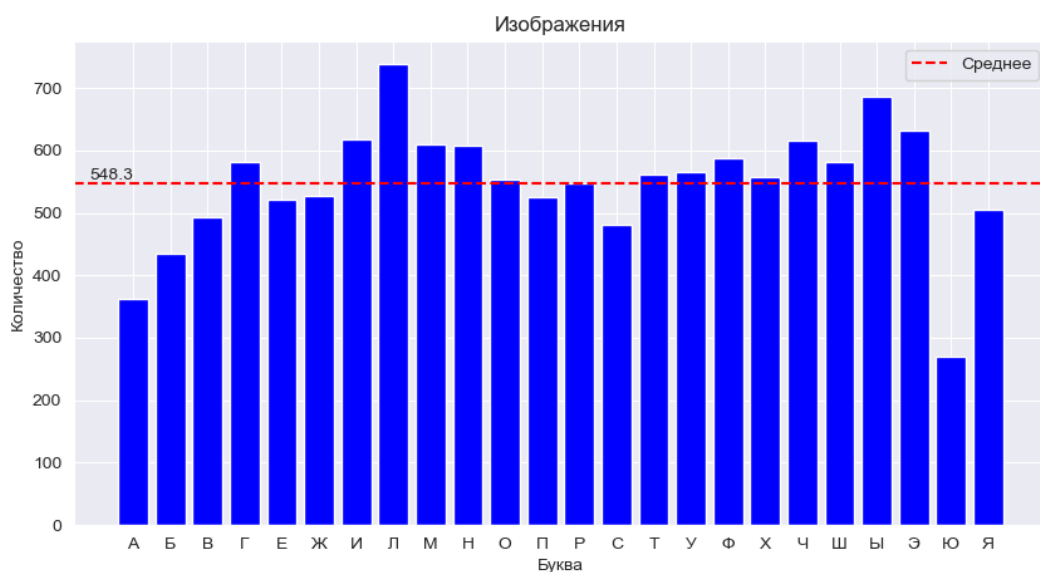


Рисунок 2.7 – Количество изображений в начальном наборе данных

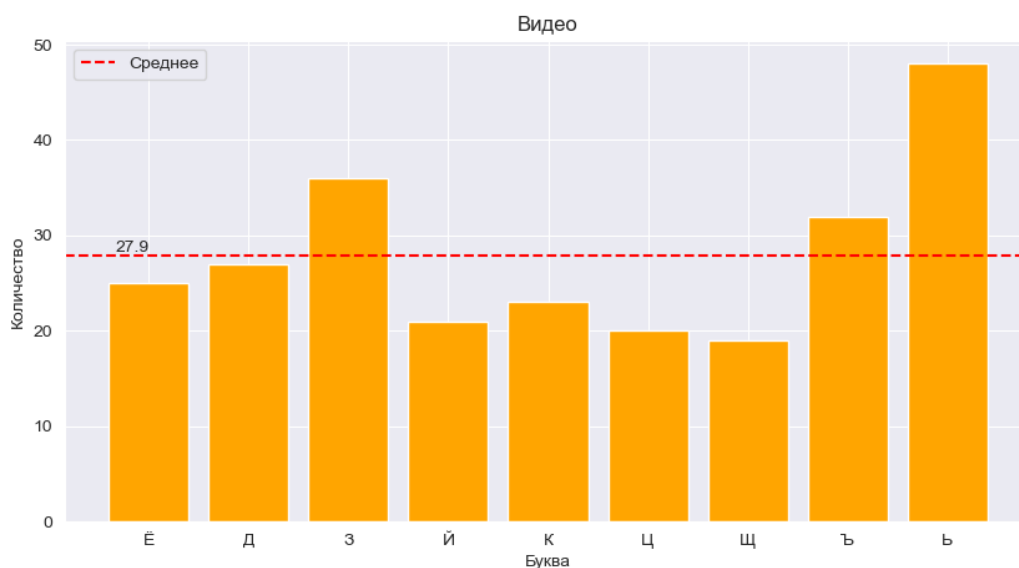


Рисунок 2.8 – Количество видео в начальном наборе данных

Как видно из диаграмм, набор данных несбалансирован, но не критично. Среднее количество изображений – 548, а среднее количество видео – 28. Одной из основных проблем данного набора данных является неоднородность данных. Для комфортной работы с данными правильным решением будет привести всё к одному типу. Так как в данной задаче важно учитывать контекст, то наилучшим решением будет привести всё в видео. Но перед переводом нужно разобраться, что из себя представляет стандартный видеофайл.

Основные характеристики видео приведены на рисунке 2.9.

Видео	
Продолжительность	00:00:01
Ширина кадра	1920
Высота кадра	1072
Скорость передачи данных	3174 кбит в сек
Общая скорость потока	3174 кбит в сек
Частота кадров	25.00 кадров/с

Рисунок 2.9 – Характеристики видео

Соответственно стандартное видео имеет разрешение 1920x1080, продолжительность в 1 с и частоту кадров 25 кадров/с. Зная эти данные не составит труда восстановить из кадров видео.

Все изображения – это просто нарезанные кадры из исходных видео. Создатели так решили упростить работу с набором данных, однако нам данный подход совсем не подходит. Стандартно название выглядит следующим образом: «номерЧеловека_номерКадра.png». Это значит, что у нас есть всё, что нужно для создания видео из набора кадров. Методика проста:

- 1) Проходим по всем кадрам определенного человека в порядке возрастания
- 2) Создаём видео
- 3) Добавляем в видео кадр, пока число кадров не равно 25
- 4) Возвращаемся к пункту 2

Таким образом удалось восстановить исходные видео, что позволило обучать необходимые нам модели. Также размер набора данных снизился примерно в 37 раз. Объем и количество файлов представлены на рисунке 2.10.

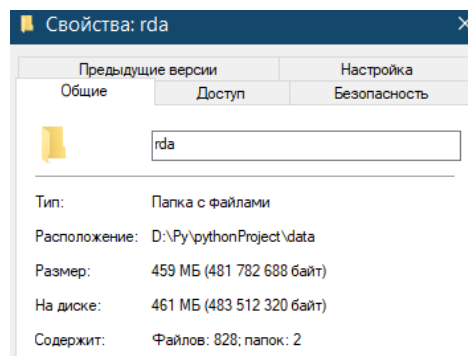


Рисунок 2.10 – Свойства итогового набора данных

Ознакомимся также со статистикой для итогового набора данных на рисунке 2.11.

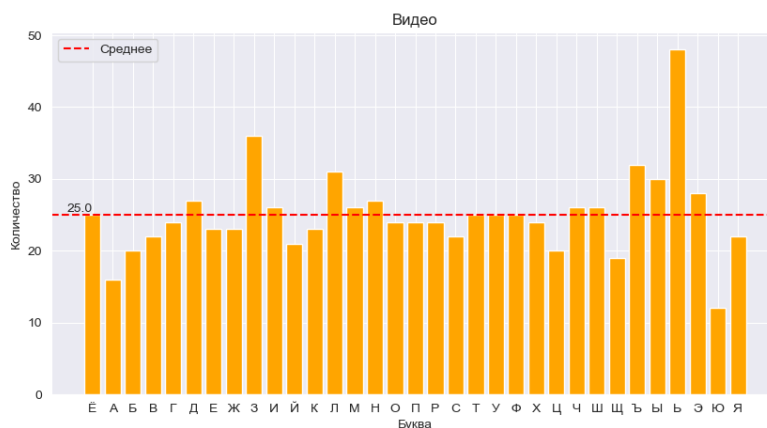


Рисунок 2.11 – Количество видео в итоговом наборе данных

Давайте рассмотрим более подробно сами кадры. Типичный кадр из видео тренировочного набора представлен на рисунке 2.12.



Рисунок 2.12 – Кадр из набора данных

Все видео сняты с одного ракурса (строго фронтально). На фоне белой стены без каких-либо визуальных шумов. На одном и том же удалении от камеры. Для сбора данных также не привлекали левшей, что не критично, ведь можно это решить правильной аугментацией данных. Также все эксперты показывают жесты исключительно в чёрной кофте. Исходя из этого можно сделать вывод, что датасет крайне тепличный и не покрывает всех возможных ситуаций, что впоследствии может сказаться на точности распознавания при других данных (например, при попытке распознать жест, снятый в комнате с большим количеством объектов, на другом удалении от камеры немного под уклоном).

В любом случае первостепенная задача заключается в том, чтобы добиться наивысших результатов на данном наборе данных, что и будет выполнено в данной ВКР.

3 ПОДБОР ПОДХОДЯЩИХ АРХИТЕКТУР НЕЙРОННЫХ СЕТЕЙ И ИХ ОБУЧЕНИЕ

В данном исследовании рассмотрено несколько архитектур нейронных сетей, каждая из которых имеет свои достоинства и недостатки. В этой главе будет детально рассказано про обработку данных, процесс обучения модели, а также про устройство самих архитектур.

Итак, архитектуры, которые предназначены для распознавания изолированных жестов, являющиеся предметом изучения в данном исследовании, можно разделить на 2 типа:

- 1) На основе ключевых точек Media Pipe
 - a. Архитектура на основе ключевых точек, использующая Random Forest
 - b. Архитектура LSTM для последовательностей ключевых точек
- 2) Нейронная сеть для анализа видео
 - a. Архитектура на основе видео, использующая визуальные трансформеры

Далее рассмотрим каждую архитектуру по отдельности в рамках отдельных подглав, посвященных процессу обучения каждой отдельной архитектуры.

3.1 Архитектура на основе ключевых точек, использующая Random Forest

Почему же мы можем использовать алгоритм Random Forest? Во-первых, Random Forest не требует больших вычислительных мощностей, что является оптимальным для подобной задачи. Random Forest также может обрабатывать большие наборы данных, а также возможна обработка многоклассовой классификации, что отлично подходит для задачи распознавания изолированных жестов.

Однако же это решение не будет идеально подходить в нашем случае, так как у него нет возможности воспринимать контекст (то есть не может учитывать взаимодействие между отдельными признаками), что в наших

условиях является огромным минусом. Хотя и простых жестов у нас большинство, у нас есть также и жесты, для которых важно и пространство времени.

Данные минусы не позволяют в итоге использовать эту архитектуру, но это не мешает нам изучить возможный вариант реализации, что и будет разобрано в дальнейшем.

Какие же действия были предприняты для обучения на Random Forest. Во-первых, работа ведётся исключительно с простыми жестами, которые представлены изображениями. Главная задача – это извлечь ключевые точки с помощью Media Pipe.

Так как в русской дактильной азбуке все жесты делаются одной рукой, то в Media Pipe поставим ограничение на количество одновременно распознаваемых рук. После этого Media Pipe будет воспринимать только 1 руку, что практически исключит ложные определения руки, а также позволит каждый раз записывать только одну руку, что нам и нужно. Далее стоит вопрос в том, как эти точки записывать. Так как Media Pipe строит модель руки по 21 ключевой точке, то их и будем записывать в массив `numru`. Каждая точка представлена 4 параметрами (X, Y, Z и параметр, отвечающий за видимость). Из каждой точки будет брать 3 наиболее важных параметра – точки X, Y, Z. Таким образом в итоге на 1 объект в нашем массиве будет 63 числа (по 3 числа на точку, общее количество точек $21 \Rightarrow 21 * 3$).

Когда появилось понимание, в каком виде будут храниться данные, нужно решить, каким образом эти данные хранить. Было принято хранить ключевые точки из нашего набора данных в файле формата `pickle`. Дело в том, что `pickle` позволяет хранить в себе почти любые объекты Python, что позволит нам хранить массивы `numru` и истинные метки классов. Для этого мы создаём 2 ключа «data» и «labels» для массива точек и меток соответственно. После мы проходим по всем директориям и записываем все ключевые точки в этот файл.

Далее используем полученные данные для обучения. Разделяем нашу выборку на `train` и `test` в соотношении 80 на 20.

Проводим обучение RandomForestClassifier из библиотеки sklearn. После обучения получившаяся модель на тренировочной выборке показывает точность 100 %, что позволяет нам сделать предположение, что модель могла переобучиться, однако предметно об этом поговорим в следующей главе.

В целом, скорость обучения является самой высокой по сравнению с другими архитектурами, рассматриваемыми в данном исследовании. Однако стоит учитывать, что количество данных, используемых для обучения немного меньше, чем в других архитектурах.

3.2 Архитектура LSTM для последовательности ключевых точек

Как упоминалось выше Random Forest неплохо справляется с классификацией статических жестов, для которых не нужно понимать контекст. Для решения этой задачи было принято решение обучить архитектуру, основанную на LSTM.

Итак, первое, над чем нужно было подумать – это каким образом нужно хранить данные о точках, ведь теперь жесты будут не только статическими, но и динамическими, что немного усложняет задачу, ведь LSTM принимает на вход строго определенное количество параметров.

Как говорилось в главе 2.4, из исходного набора данных получилось сделать набор данных, который включает в себя только видео строго до 25 кадров. Так как максимальное количество кадров в нашем видео = 25, то будем подавать на вход модели ключевые точки из 25 кадров. Осталась лишь одна проблема – в нашем наборе данных есть видео, в которых количество кадров меньше 25. В этом случае было решено применить padding – то есть занулить недостающие параметры.

Так как нас интересуют только ключевые точки, то из каждого видео достанем их. В отличие от архитектуры, основанной на Random Forest, в данном случае решено из каждого видео сохранить двумерный массив numpy, где покадрово хранятся ключевые точки руки. Если количество кадров меньше 25, то остальные элементы двумерного массива заполняются массивом из 21

нуля. Данные двумерные массивы решено хранить в файлах формата .npy. Для легкого использования во время обучения.

Так же, как и в случае с Random Forest данные разделены в соотношении 80 на 20 на тренировочную и тестовую.

Следующей задачей была разработка архитектуры нейронной сети на основе LSTM. Было принято решение разработать последовательную модель, то есть модель, в которой слои будут соединены друг за другом. Архитектура реализовывалась с помощью с помощью Tensorflow[11] – библиотеки для машинного обучения, разработанная компанией Google.

Описание каждого слоя архитектуры представлено на таблице 2.

Таблица 2 – Слои архитектуры нейронной сети на основе LSTM

Слой	Кол-во нейронов	Функция активации
LSTM	64	ReLU
LSTM	128	ReLU
LSTM	64	ReLU
Dense	64	ReLU
Dense	32	ReLU
Dense	33	softmax

Также помимо слоёв, представленных в таблице также есть входной слой. Входной слой принимает данные размером (25, 21*3). 25 – это количество кадров в последовательности, а 21*3 – общее количество точек руки (всего 21 точка, каждая точка в 3 проекциях).

Первый слой нейронной сети – это LSTM, с 64 скрытыми нейронами, слой возвращает полную последовательность выходных данных. Второй слой нейронной сети также LSTM, с 128 скрытыми нейронами, так же, как и предыдущий возвращает всю последовательность данных. Третий слой – LSTM с 64 скрытыми нейронами, который возвращает последний выход. Далее идут полносвязные слои с 64 и 32 нейронами. Последний полносвязный слой имеет 33 нейрона (по кол-ву классов в данных). А функция активации softmax – для определения класса.

Обучалась же модель на 350 эпохах. Так как у нас задача многоклассовой классификации, то для оценки точности и потерь выбрана категориальная метрика.

По итогу обучения удалось достичь точности на тренировочной выборке в 97 %, что является отличным результатом, однако точно говорить о результатах обучения можно будет на этапе экспериментальной проверки полученной модели.

Также в контексте оценки производительности важно изучить график изменения функций точности и потерь.

Функция потерь – это мера того, насколько предсказания модели отличаются от истинных значений. Во время обучения важно, чтобы функция потерь постепенно уменьшалась.

Функция точности – это процент правильных предсказаний, которые делает модель. Во время обучения важно, чтобы функция точности постепенно увеличивалась.

Графики изменения функции точности и потерь представлены на рисунке 3.1 и рисунке 3.2.

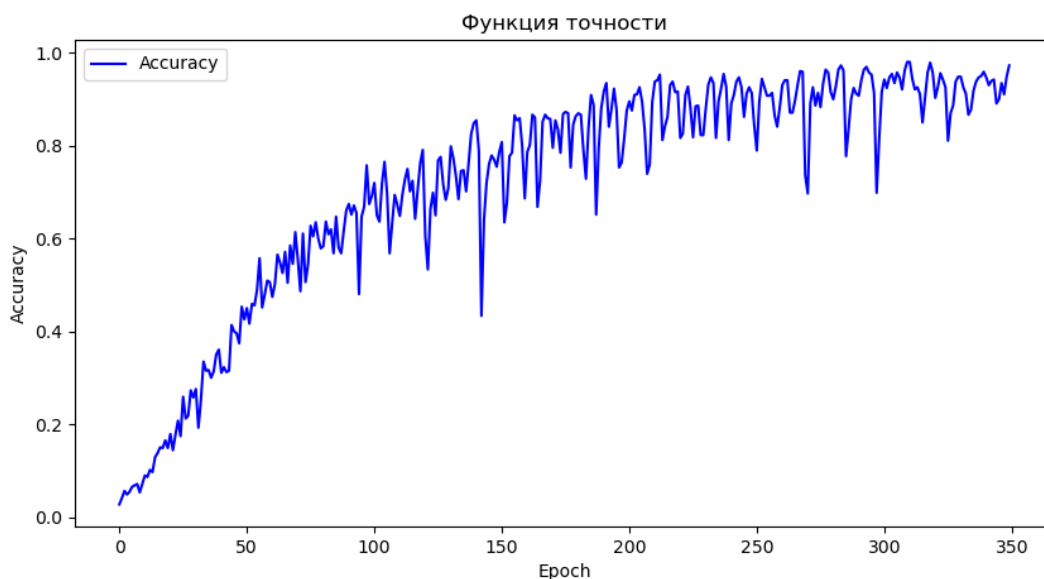


Рисунок 3.1 – Функция точности модели LSTM

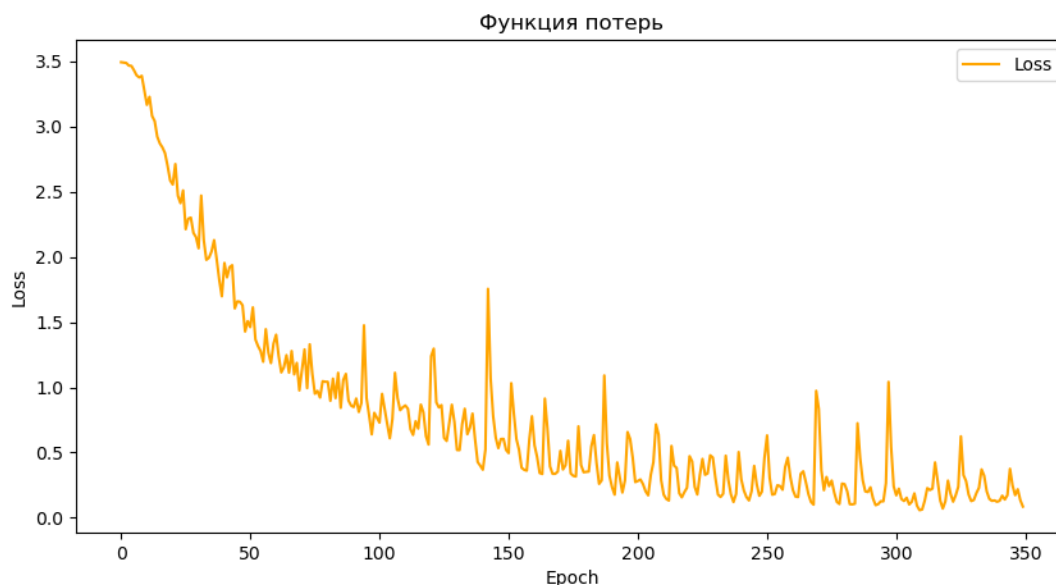


Рисунок 3.2 – Функция потерь модели LSTM

Как видно на графиках, обучение прошло достаточно успешно. Для LSTM характерно, когда на графиках функций точности и потерь появляются скачки. В целом, функция потерь с каждой эпохой снижается, а функция точности увеличивается.

Скорость обучения модели всё ещё достаточно высокая, так же, как и производительность модели.

3.3 Архитектура на основе видео, использующая визуальные трансформеры

Итак, одной из главных задач данной работы было обучение именно визуального трансформера. Существующих архитектур достаточно много неясно, какую архитектуру стоит выбрать.

Как и в исследовании [7], было принято решение использовать наиболее распространенную архитектуру в задаче распознавания видео – Video Swin Transformer [8].

Одна из задач, которая ставилась исследованию [7] – обучить такую архитектуру, которая могла бы распознавать жесты с неплохой производительностью на обычном ноутбуке. Для этого они выбрали самую легкую архитектуру, которая называется Swin-T. После обучения данной модели авторы статьи проверили её работу на обычном ноутбуке, который смог

запустить задачу распознавания жестов при этом с достаточно хорошей производительностью в 3 предсказания в секунду. Поэтому было принято решение использовать данную архитектуру, ведь задачи, стоящие в нашем исследовании схожи.

Первой проблемой, с которой пришлось столкнуться – как обучить данную архитектуру. Для этой задачи было принято решение использовать фреймворк MMAction2[10], который предоставляет гибкие инструменты для обучения моделей множества задач распознавания действий.

После изучения данного фреймворка стало ясно, как нужно организовать набор данных, чтобы обучение прошло успешно.

Новый набор данных был разделен на тренировочную и валидационную выборки в соотношении 80 на 20. В данном случае тестовая выборка аналогична валидационной. Также были созданы файлы аннотаций для обеих выборок. В них хранятся названия файлов и истинные классы каждого объекта. Также был создан label map – хранящий названия всех классов.

После того, как был организован набор данных, осталось написать конфигурационный файл, в котором будут прописаны все нюансы нашей модели.

Перечислим основные параметры, которые задаются в конфигурационном файле:

- 1) Модель будет проинициализирована весами, предварительно обученными на наборе данных Kinetics-400. Это помогает увеличить производительность модели в условиях ограниченности данных
- 2) Все видео нормализуются для улучшения обучения модели
- 3) Для каждого батча будет применен случайный набор аугментаций. В данном случае будут применяться MixupBlending, который смешивает 2 образца данных и их метки, и CutmixBlending, который смешивает 2 образца данных, заменяя часть одного образца на часть другого. Всё это поможет увеличить обобщающую способность модели

4) Каждое видео меняет размер на 224x224

Стоит отметить, что трансформеры в принципе требуют больших вычислительных мощностей, поэтому обучение данной модели производилось на вычислительном кластере факультета, имеющего мощные вычислительные ресурсы. Однако даже с этим условием на 2-ух видеокартах получилось обучать модель при размере батча равным 5.

Для того, чтобы воспользоваться вычислительным кластером был разработан docker-образ для задачи обучения модели. Также в процессе обучения на кластере возникла проблема с размером общей памяти, выделявшейся для каждого docker-контейнера – это удалось решить после разговора с администратором, который увеличил размер общей памяти для межпроцессного взаимодействия в поде.

После решения возникших проблем появилась возможность обучения модели.

В результате обучения получилось достичь 95% точности на валидационной выборке, что является очень хорошим результатом, однако подробнее рассмотрим на этапе экспериментальной проверки полученной модели.

Также рассмотрим детальнее графики функций точности и потерь, которые представлены на рисунке 3.3 и рисунке 3.4.



Рисунок 3.3 – Функция точности модели Video Swin Transformer

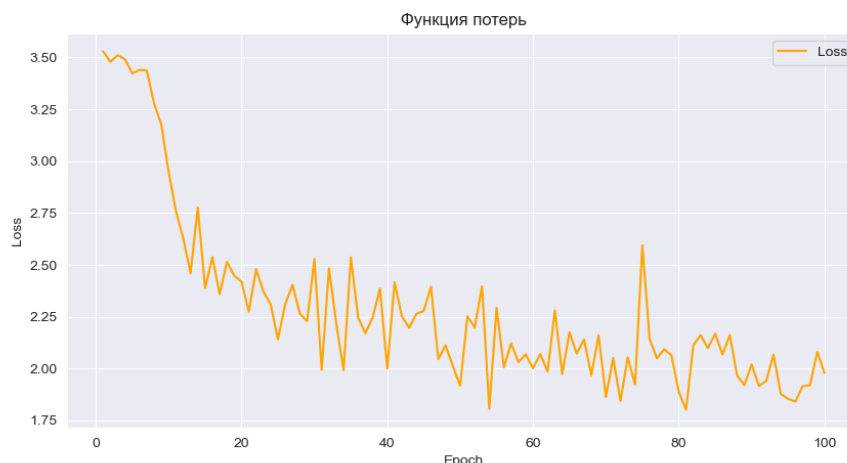


Рисунок 3.4 - Функция потерь модели Video Swin Transformer

Стоит учитывать, что графику функции потерь для трансформеров свойственна скачкообразность. Как видно из графика точности – модель достаточно быстро смогла отнести в 5 предполагаемых жестов истинный жест, который показывался. Также в процессе обучения на 100 эпохах точность постепенно увеличилась и в итоге достигла 95%. Более подробная информация о производительности модели будет рассказано на этапе экспериментальной проверки обученной модели.

4 РАЗРАБОТКА КОНВЕЙЕРА РАСПОЗНАВАНИЯ ЖЕСТОВ

В основе реализации конвейера лежит модель, выбранная по результатам проведенных экспериментов, описанных в главе 5.

Задача разработки конвейера распознавания жестов очень важна, так как от не зависит будущая производительность системы в целом, ведь не стоит забывать, что задача распознавания изолированных жестов – это лишь одна из трех задач, которые в перспективе могут быть объединены в один ML-конвейер. Таким образом производительность каждой модели будет играть огромную роль для создания системы, которая должна быть не требовательна к вычислительным ресурсам.

Итак, первое, что следовало решить, при создании ML-конвейера – это его входные и выходные данные.

На вход конвейер принимает следующие данные:

- 1) Путь к входному видео
- 2) Путь к выходному видео
- 3) Путь к выходному текстовому файлу
- 4) Путь к конфигурационному файлу модели
- 5) Путь к файлу модели
- 6) Порог вероятности предсказанного действия (Опционально)

Данные входные параметры обеспечивают гибкость в использовании конвейера, позволяя адаптировать его под различные задачи и условия.

Выходные данные:

- 1) Обработанное видео
- 2) Список предсказанных жестов

Видео представляет из себя обработанное исходное видео, с отображением предсказанных жестов. Пример кадров из обработанного видео представлен на рисунке 4.1.

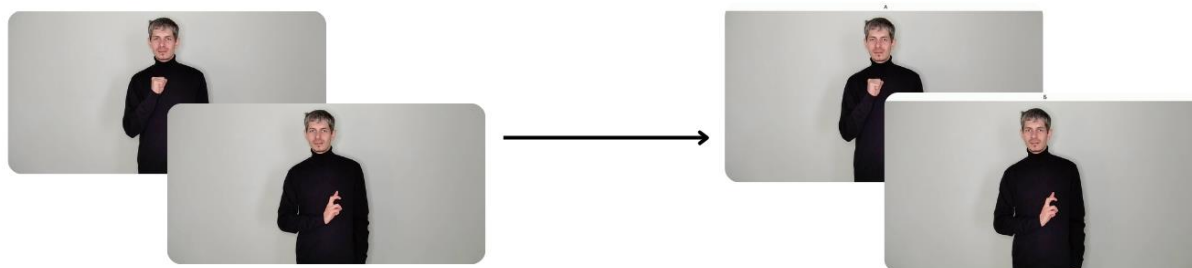


Рисунок 4.1 – Процесс преобразования входного в конвейер видео

Далее перейдем к логике функционирования конвейера. Далее перечислим ключевые действия, которые выполняет конвейер:

- 1) Загрузка модели Swin Video Transformer с помощью функции `init_recognizer()`, которая доступна в фреймворке `maction2`
- 2) Открытия видеофайла и последующая покадровая обработка
- 3) Каждая последовательность из 25 кадров подаётся на вход в модель
- 4) Если вероятность предсказанного жеста выше заданного порога, то запись обозначения жеста (буквы) в текстовый файл и на видео
- 5) Сохранение обработанного видео в новый файл

Так как данное исследование предполагается расширять, а для дальнейших исследований требуется много вычислительных ресурсов, то было решено подготовить данное решение к запуску на сервере, поэтому был создан докер-образ данного конвейера, после чего появилась возможность запуска данного программного решения на сервере.

5 ЭКСПЕРИМЕНТАЛЬНАЯ ПРОВЕРКА

В данной главе представлены результаты экспериментальной проверки обученных моделей. Это важно для оценки точности и эффективности представленных в исследовании моделей. Во время экспериментов был использован исходный набор данных. Также будут представлены:

- 1) Процессы подготовки данных для тестирования
- 2) Процессы тестирования моделей
- 3) Анализ полученных результатов

Так же, как и в прошлой главе, для каждой архитектуры будет описан процесс экспериментальной проверки.

5.1 Архитектура на основе ключевых точек, использующая Random Forest

Итак, для тестирования, как говорилось ранее, используется 20% первоначального набора данных. Никакой предварительной обработки данных не производится.

Тестируется данная модель на изображениях, так как данная архитектура ограничена тем, что не способна воспринимать контекст.

Само процесс тестирования заключается в том, что модели на вход подаются изображения, она делает свои предсказания, после чего значения сравниваются с истинными.

В итоге тестирования данной модели получилось достичь точности в 100%, однако это обусловлено тем, что изображения нарезались из видео, в которых эксперты были неподвижными, поэтому точки изменялись не критично. Однако пока было не ясно, переобучилась ли модель, или же выдаёт идеальные результаты. Было решено взять тестовое видео, предложенное разработчиками датасета (оно, к сожалению, не размечено), посмотреть за предиктами, которые делает наша модель.

Для распознавания жестов в реальном времени было решено брать каждый кадр из видео (может быть как какое-то видео извне, так и видео с веб-

камеры), после чего доставать из кадра набор ключевых точек, после чего подавать набор ключевых точек на вход в модель.

Таким образом появилась возможность получать данные о предсказаниях в реальном времени. После этого было решено проверить видео.

В итоге оказалось, что многие жесты распознаются корректно, однако о точности в 100%, хотя и на самых простых жестах, говорить не стоит. У модели даже в самых простых ситуациях есть склонность ошибаться, если точки рук находятся не в том же месте, что и в исходных видео.

Пример того, как производилась экспериментальная проверка модели вручную представлена на рисунке 5.1.

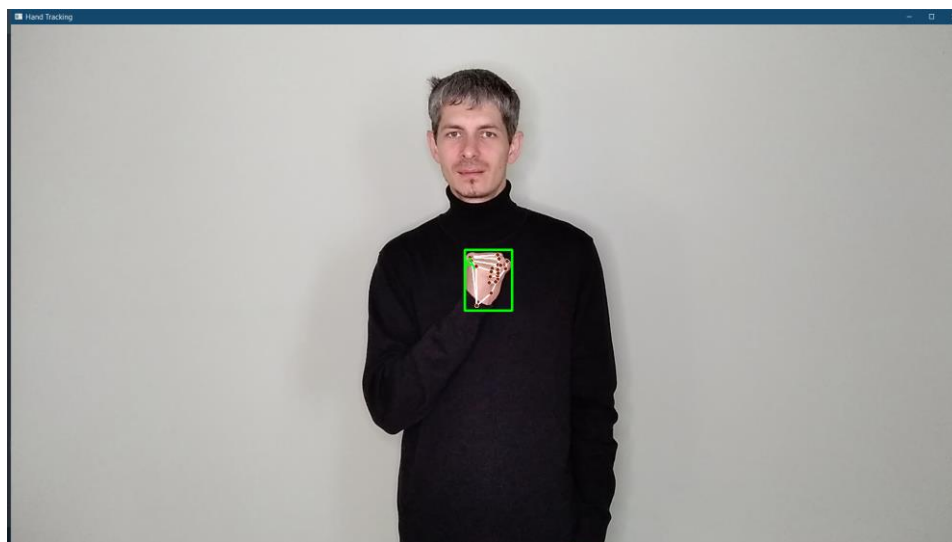


Рисунок 5.1 – Проверка модели Random Forest вручную

По итогам экспериментальной проверки данной модели стало окончательно ясно, что данная архитектура для нашей задачи распознавания всех жестов русской дактильной азбуки не подходит.

5.2 Архитектура LSTM для последовательности ключевых точек

Тестовыми данными в данном случае являются 20% набора данных, созданного из начального набора. Каждый тестовый пример представляет из себя 25 кадров, где каждый кадр представляет из себя набор из 21 ключевой точки в 3 проекциях, что модель и принимает на вход.

Точность, которой удалось достичь на тестовой выборке, равняется 86%, что является неплохим результатом, однако наш набор данных не является сбалансированным, поэтому было принято решение посчитать матрицу ошибок, полноту и точность (confusion matrix, precision, recall), а с помощью этих метрик посчитать f1-меру. Более подробно о метриках будет рассказано ниже.

Для упрощения понимания, что такое матрица ошибок, предлагается изучить рисунок 5.2.

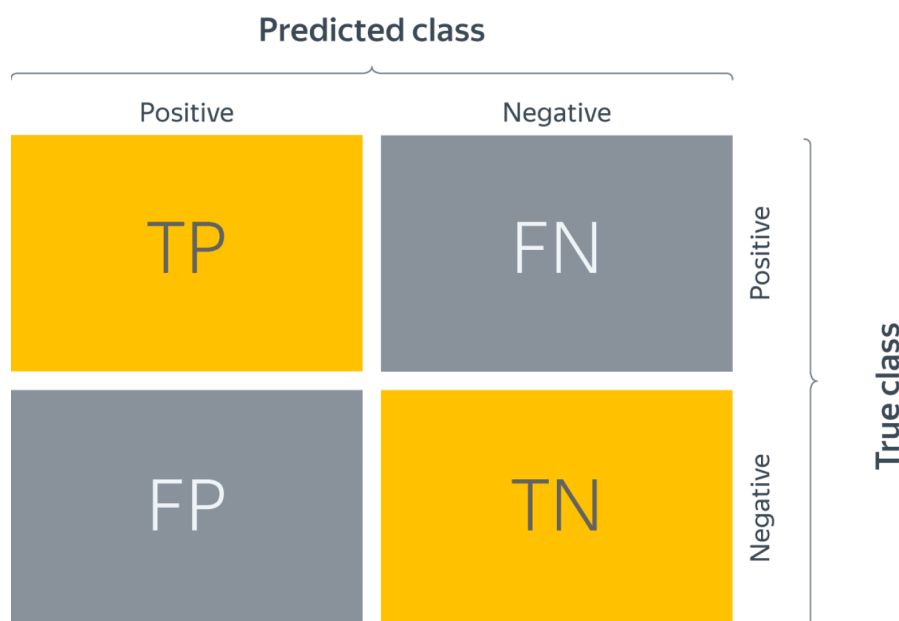


Рисунок 5.2 – Матрица ошибок

Заметим, что каждый классифицированный объект в выборке можно отнести к одной из 4 групп:

- 1) True Positive (TP) – это количество объектов, которых модель верно отнесла к данному классу
- 2) False Positive (FP) – это количество объектов, которых модель неверно отнесла к данному классу
- 3) True Negative (TN) – это количество объектов, которых модель верно отнесла к другому классу
- 4) False Negative (FN) – это количество объектов, которых модель неверно отнесла к другому классу

Как выглядит матрица ошибок для нашей тестовой выборки можно ознакомиться на рисунке 5.3.

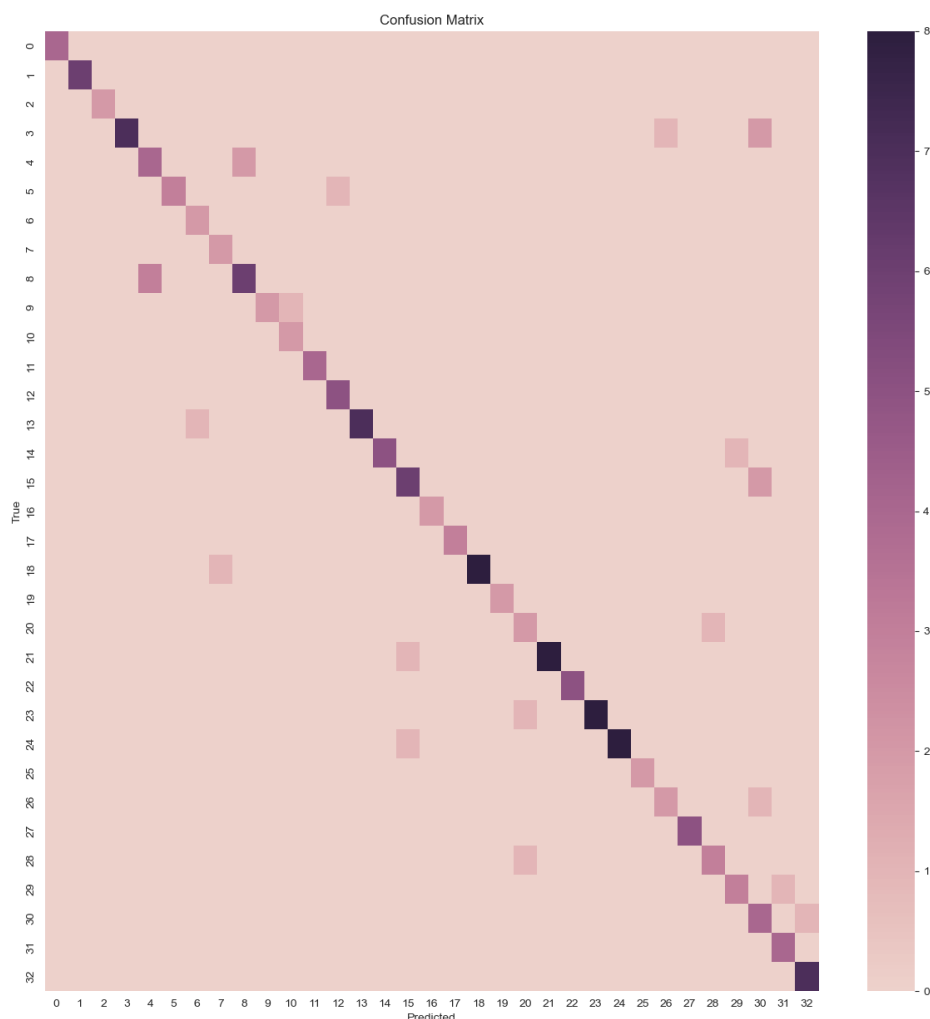


Рисунок 5.3 – Матрица ошибок архитектуры LSTM

На первый взгляд матрица ошибок выглядит неплохо, с её помощью рассчитаем ещё 3 метрики, которые скажут нам больше о качестве модели.

Precision (точность) — это доля правильно предсказанных объектов данного класса среди всех объектов, отнесенных к этому классу. Формула точности представлена ниже.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad (6)$$

Recall (полнота) – это доля правильно предсказанных объектов данного класса среди всех объектов данного класса. Формула полноты представлена ниже.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (7)$$

Однако данные формулы представлены для бинарной классификации. Для многоклассовой классификации применим макроусреднение, которое подразумевает подсчёт precision и recall для каждого класса, после чего считается среднее арифметическое. Тогда итоговые формулы для каждой метрики выглядят следующим образом:

$$\text{Precision}_{\text{macro}} = \frac{1}{|C|} \sum_{c \in C} \text{Precision}_c, \quad (8)$$

$$\text{Recall}_{\text{macro}} = \frac{1}{|C|} \sum_{c \in C} \text{Recall}_c, \quad (9)$$

Для данной модели значения $\text{Precision}_{\text{macro}} = 0.869$ и $\text{Recall}_{\text{macro}} = 0.882$, что является неплохими результатами. Чтобы обобщить пару Precision-Recall возьмём их среднее гармоническое, что называется f1-мерой.

Уравнение f1-меры представлено ниже:

$$F_1 = \frac{2}{\frac{1}{\text{Recall}_{\text{macro}}} + \frac{1}{\text{Precision}_{\text{macro}}}} = 2 \frac{\text{Precision}_{\text{macro}} * \text{Recall}_{\text{macro}}}{\text{Precision}_{\text{macro}} + \text{Recall}_{\text{macro}}}, \quad (10)$$

Для данной модели значение $F_1 = 0.876$, что говорит о том, что данная модель неплохо справляется с задачей классификации жестов, может быть применена в дальнейшем как одна из составных частей ML-конвейера.

5.3 Архитектура на основе видео, использующая визуальные трансформеры

Тестовые данные, как и в прошлых случаях – это 20% набора данных, однако в отличие от предыдущих архитектур, на вход модели подаются видео, набор преобразований для каждого видео описан в конфигурационном файле. Основные преобразования для каждого видео приведены в главе 2.2.3.

Метрики качества модели будут сходны с метриками качества, которые мы использовали для оценки модели LSTM. То есть были посчитаны матрица ошибок, precision, recall, f1-мера.

Уже невооруженным взглядом видно, что результаты у данной модели выше, чем у предыдущей, так как большинство предсказаний расположено по диагонали матрицы. Ошибочных же случаев не так много.

$\text{Precision}_{\text{macro}} = 0.939$, что уже выше примерно на 7%, чем у нейронной сети с архитектурой LSTM.

$\text{Recall}_{\text{macro}} = 0.933$, что выше примерно на 5%, чем у нейронной сети с архитектурой LSTM.

$F_1 = 0.936$, что также выше примерно на 6%.

То есть в общем и целом, архитектура на основе визуальных трансформеров показывает себя лучше, чем архитектура, использующая в своей основе LSTM.

На рисунке 5.4 изображена матрица ошибок для модели Swin Video Transformer.

Давайте рассмотрим те жесты, которые наша модель классифицирует неверно. Из матрицы ошибок видно, что модель часто путает 12 и 13 жесты (13 и 14 буквы), 16 и 19 жесты (17 и 20 буквы), а также 25 и 26 жесты (26 и 27 буквы).

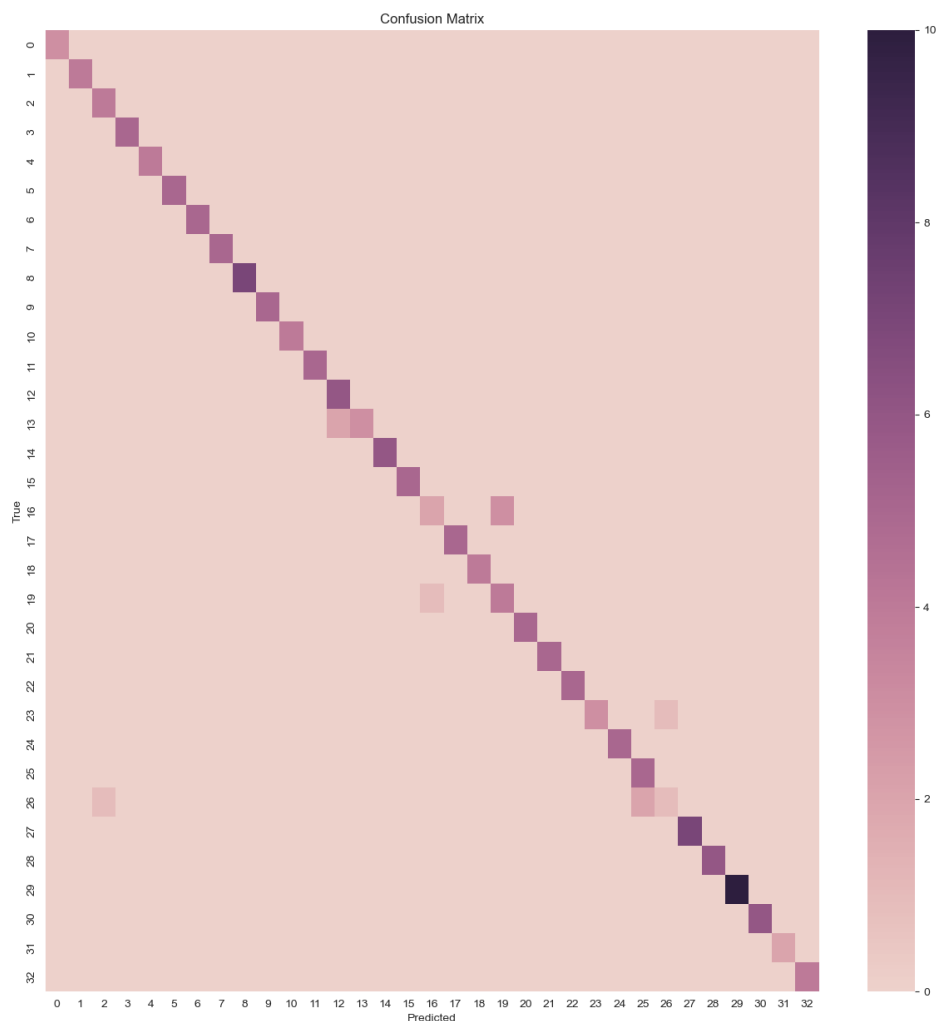


Рисунок 5.4 – Матрица ошибок для Video Swin Transformer

Если посмотреть на изображения этих жестов, то действительно визуально они бывают очень похожи, а иногда и не отличимы без контекста. Это продемонстрировано на рисунке 5.5. При этом заметим, что только жест «Щ» — это жест, который требует контекст (то есть является сложным). Это можно решить, выделив большее количество времени на обучение, а также можно увеличить обучающую выборку, например за счёт набора данных, описанного в статье [12] .

Из приведенной экспериментальной проверки каждой модели было принято решение использовать модель на основе Video Swin Transformer, как основную.

В последней главе, посвященной разработке конвейера распознавания жестов, будет описан конвейер распознавания жестов, в основе которого лежит модель на основе архитектуры Video Swin Transformer.

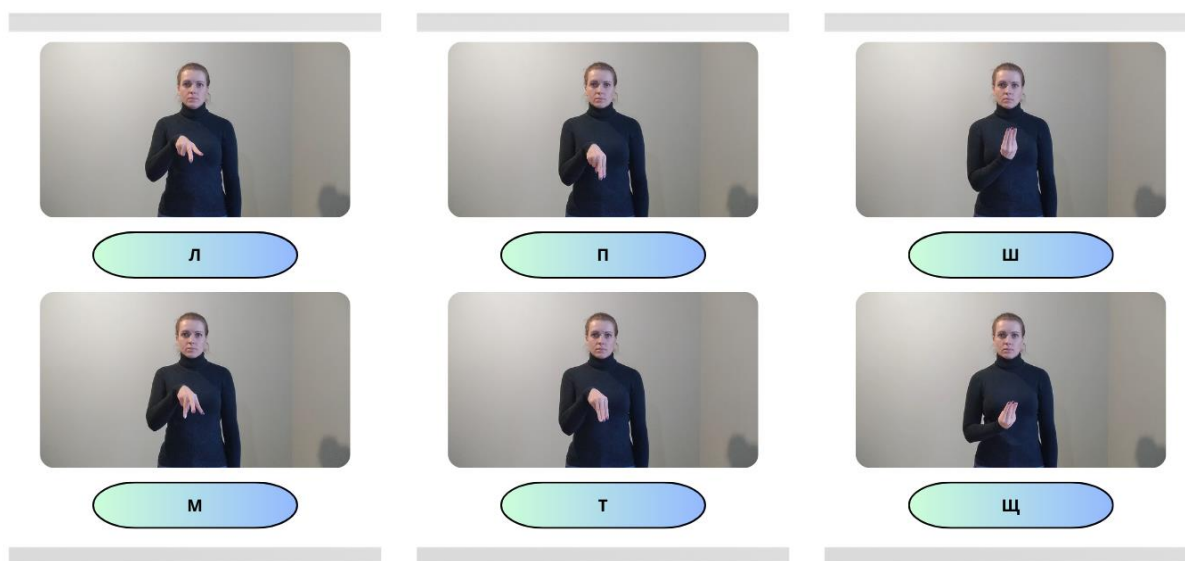


Рисунок 5.5 – Жесты, которые путает модель Video Swin Transformer

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломной работы были успешно решены все поставленные задачи, а именно:

- 1) Был проведен анализ предметной области и ограничений набора данных дактильной азбуки. Это позволило выбрать подходящие архитектуры для достижения поставленной цели, также способствовало более глубокому пониманию проблемы
- 2) Были подобраны и обучены подходящие архитектуры нейронных сетей – это дало множество альтернативных вариантов с разными плюсами и минусами, которые были выявлены на этапе экспериментальной проверки обученной нейронной сети
- 3) Были экспериментально проверены все полученные модели. Для каждой модели была приведена статистика и подобраны метрики, которые позволили объективно оценить качество полученных моделей
- 4) Был разработан конвейер распознавания жестов, который позволил автоматизировать процесс распознавания жестов и значительно упростить использование полученной модели

Таким образом цель дипломной работы была достигнута. Была разработана нейронная сеть, способная с высокой точностью распознавать жесты русской дактильной азбуки. Это позволяет активно использовать данное решение в рамках полного цикла распознавания жестов, который был описан ранее, а также данная модель может использоваться как основа для других моделей, основанных на Swin Video Transformer.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Real-Time Hand Gesture Recognition Using Fine-Tuned Convolutional Neural Network / P. S. Jaya. — Текст : электронный // ncbi.nlm.nih.gov : [сайт]. — URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8840381/> (дата обращения: 20.04.2024).
2. Agrawal, S. Real-Time Hand Gesture Recognition System Using MediaPipe and LSTM / S. Agrawal, A. Chakraborty, M. Rajalakshmi. — Текст : непосредственный // International Journal of Research Publication and Reviews. — 2022. — № 4. — С. 2509-2515.
3. Attention Is All You Need / Vaswani. — Текст : электронный // arxiv.org : [сайт]. — URL: <https://arxiv.org/abs/1706.03762> (дата обращения: 20.04.2024).
4. An image is worth 16x16 words: transformers for image recognition at scale / Dosovitskiy. — Текст : электронный // arxiv.org : [сайт]. — URL: <https://arxiv.org/abs/2010.11929> (дата обращения: 25.04.2024).
5. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding / Devlin. — Текст : электронный // arxiv.org : [сайт]. — URL: <https://arxiv.org/abs/1810.04805> (дата обращения: 25.04.2024).
6. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows / Liu Ze. — Текст : электронный // arxiv.org : [сайт]. — URL: <https://arxiv.org/abs/2103.14030> (дата обращения: 26.04.2024).
7. Fine-tuning of sign language recognition models: a technical report / Novopoltsev Maxim. — Текст : электронный // arxiv.org : [сайт]. — URL: <https://arxiv.org/abs/2302.07693> (дата обращения: 07.05.2024).
8. Video Swin Transformer / Liu Ze. — Текст : электронный // arxiv.org : [сайт]. — URL: <https://arxiv.org/abs/2106.13230> (дата обращения: 07.05.2024).
9. Руководство по решениям MediaPipe. — Текст : электронный // ai.google.dev : [сайт]. — URL:

- <https://ai.google.dev/edge/mediapipe/solutions/guide> (дата обращения: 18.05.2024).
10. MMAction2 documentation. — Текст : электронный // mmdetection.readthedocs.io : [сайт]. — URL: <https://mmdetection.readthedocs.io/en/latest/index.html> (дата обращения: 18.05.2024).
11. Tensorflow API documentation. — Текст : электронный // [tensorflow.org](https://www.tensorflow.org/api_docs) : [сайт]. — URL: https://www.tensorflow.org/api_docs (дата обращения: 18.05.2024).
12. Slovo: Russian Sign Language Dataset / Kapitanov Alexander. — Текст : электронный // [arxiv.org](https://arxiv.org/abs/2305.14527) : [сайт]. — URL: <https://arxiv.org/abs/2305.14527> (дата обращения: 18.05.2024).
13. OpenCV documentation. — Текст : электронный // docs.opencv.org : [сайт]. — URL: <https://docs.opencv.org/4.x/> (дата обращения: 18.05.2024).